

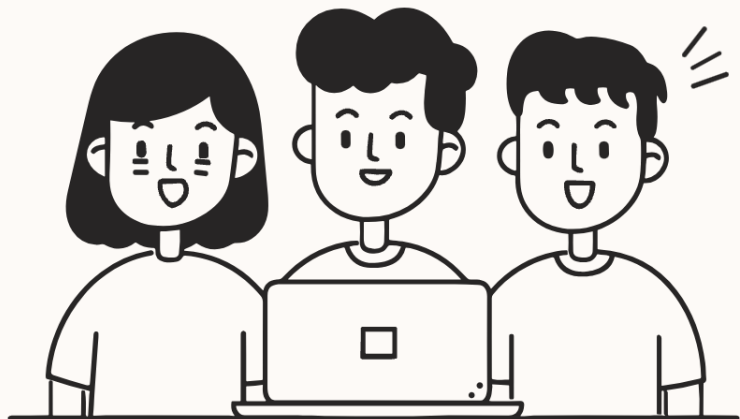
Как практиковаться начинающему DevOps- инженеру

Практический маршрут от виртуальной машины и Linux до Docker, мониторинга и Kubernetes.



Зачем нужна практика

Junior DevOps важно руками собирать окружение, запускать сервисы, ломать их, чинить, читать логи и фиксировать выводы.



Собирать

Настраивать окружение с нуля



Ломать

Намеренно воспроизводить сбои



Читать логи

Находить причины ошибок



Фиксировать

Записывать выводы и решения

Совет перед началом

Вести заметки, записывать команды, ошибки, решения и сравнивать инструменты между собой.

Команды

Записывайте каждую команду, которую вводите в терминал

Ошибки

Фиксируйте сообщения об ошибках и способы их устранения

Решения

Описывайте, что сработало и почему

Сравнение

Сопоставляйте инструменты между собой по удобству и возможностям



Темы практики

Полная карта навыков начинающего DevOps-инженера — от базовой инфраструктуры до оркестрации.



Виртуальный сервер

VPS, гипервизор, ОС



SSH

Ключи, подключение, туннели



Linux

Пакеты, сеть, права, диагностика



Проект

HTTP-сервис, JSON, порт



systemd

Unit-файл, автозапуск



Автоматизация

GitLab CI, Jenkins, runner



Docker

Образы, контейнеры, Dockerfile



Registry

Docker Hub, Nexus, private



Docker Compose

Многоконтейнерные приложения



nginx

Reverse proxy, SSL, HTTPS



Мониторинг

Prometheus, Zabbix, алерты



Логирование

ELK, Loki, запросы



Документация

Развёртывание, эксплуатация



Kubernetes

Minikube, K3s, ingress

Создание виртуальной машины



Варианты развёртывания

Что настроить

- Выбор Linux-дистрибутива (Ubuntu, Debian, CentOS)
- Установка операционной системы
- Настройка дисков и разделов
- Конфигурация сети



Локальный гипервизор

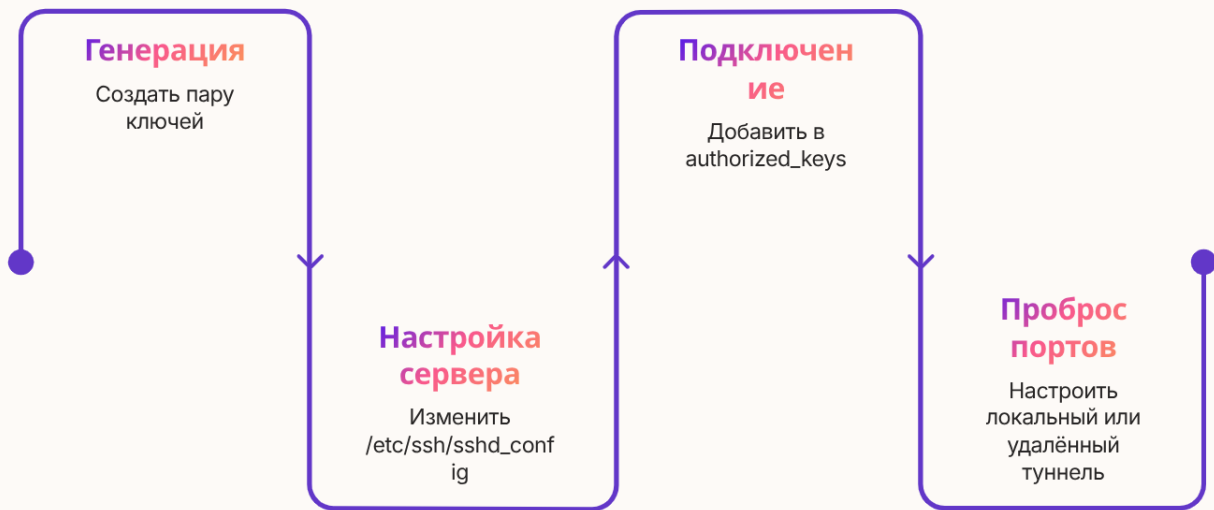
VirtualBox, VMware, KVM



VPS

Облачный виртуальный сервер

SSH



Ключевые темы

- **Ключи** — генерация пары публичный/приватный ключ
- **SSH-клиент** — настройка `~/.ssh/config`
- **Конфигурация сервера** — `/etc/ssh/sshd_config`
- **Подключение без пароля** — `authorized_keys`
- **Проброс портов** — локальный и удалённый туннели

Настройка Linux



Обновления

apt update, apt upgrade, yum



Пакеты

Установка и управление ПО



Файловая система

Структура, монтирование, df, du



Диагностика

top, htop, ps, lsof, netstat



Сеть

ip, ifconfig, ping, traceroute



Пользователи и права

useradd, chmod, chown, sudo

Проект для практики



Что представляет собой проект

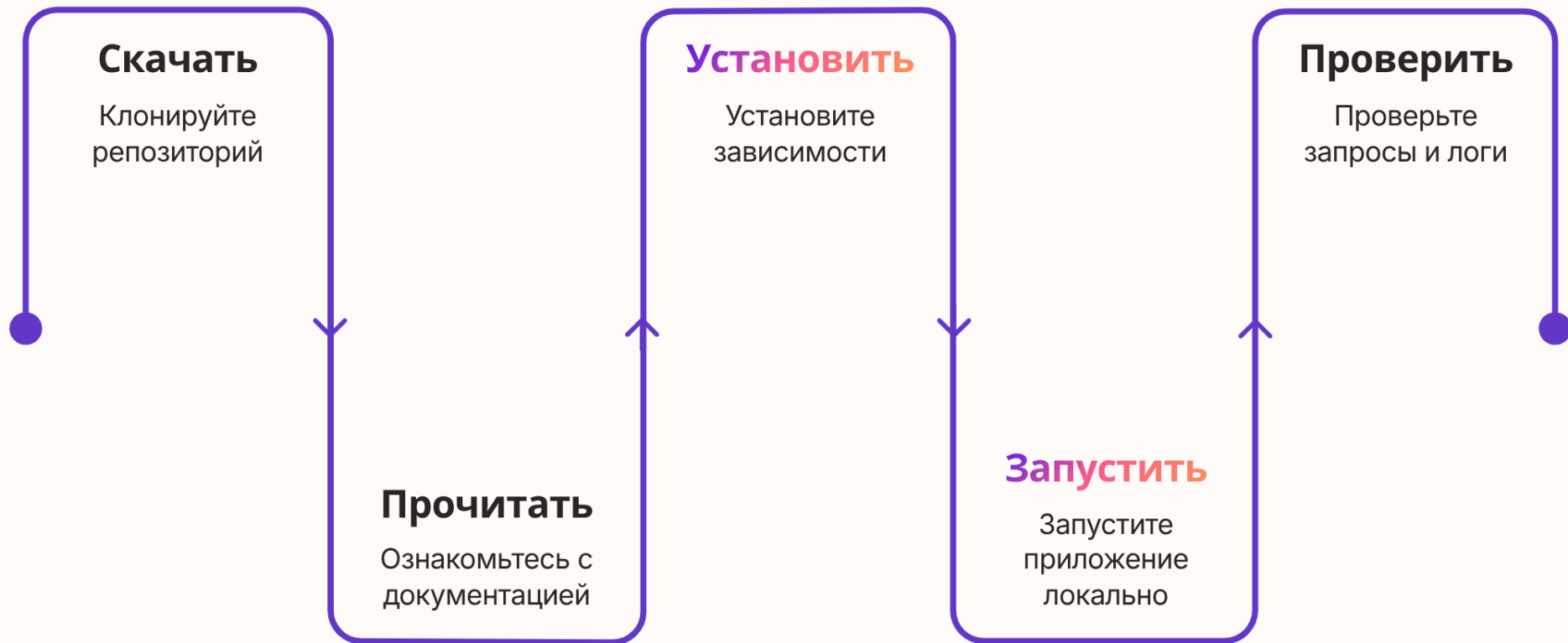
Простое приложение, которое слушает порт, отвечает на HTTP-запрос и возвращает JSON.

- ❏ Идеальный учебный проект — минимальный, но реалистичный. Он должен иметь зависимости, конфигурацию и логи.

Примеры стека

- Python + Flask / FastAPI
- Node.js + Express
- Go + net/http

Ручной запуск проекта



Ручной запуск — это первый шаг перед автоматизацией. Важно понять каждый этап, прежде чем передавать его скриптам или CI/CD-системе.

systemd

Unit-файл

/etc/systemd/system/
myapp.service

Переменные окружения

EnvironmentFile,
Environment=

systemctl

start, stop, restart,
enable

Автозапуск

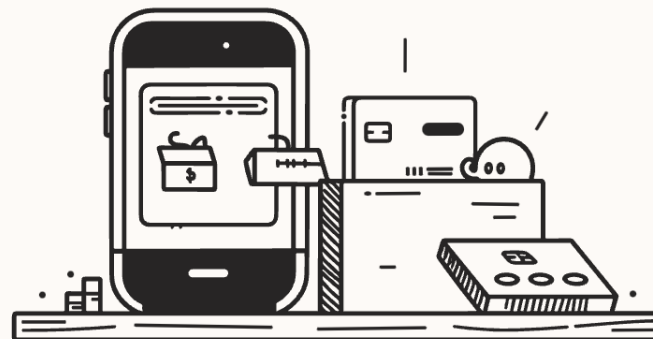
WantedBy=multi-
user.target

status

systemctl status
myapp

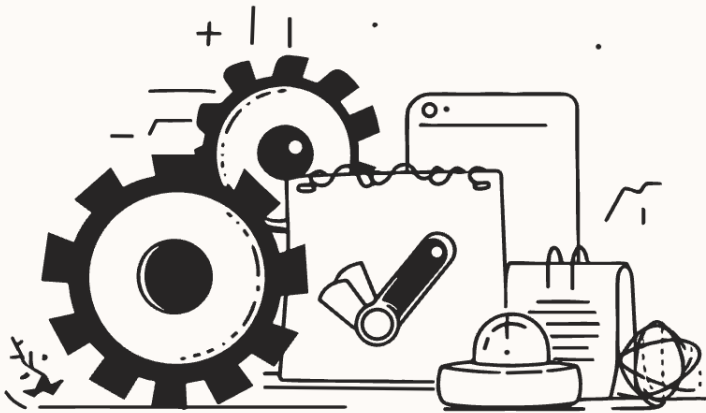
journalctl

journalctl -u myapp -f



Автоматизация

GitLab CI/CD или Jenkins — основа автоматизации в DevOps. Runner выполняет задачи pipeline, а сам pipeline описывает шаги сборки и деплоя приложения.



GitLab / Jenkins

Платформы для CI/CD:
хранение кода и
автоматизация процессов



Runner

Агент, который выполняет
задачи pipeline на сервере или
в контейнере



Pipeline

Цепочка шагов: сборка → тест → деплой, запускается
автоматически

Контейнеризация

Контейнер — изолированная среда запуска приложения. Образ состоит из слоёв, каждый из которых кэшируется отдельно. Dockerfile описывает, как собрать образ.

- **Dockerfile** — инструкция по сборке образа
- **Сборка образа** — `docker build -t myapp .`
- **Запуск контейнера** — `docker run -d -p 8080:80 myapp`
- **Порты** — проброс портов хоста в контейнер
- **Volumes** — монтирование данных с хоста в контейнер

Образ (Image)

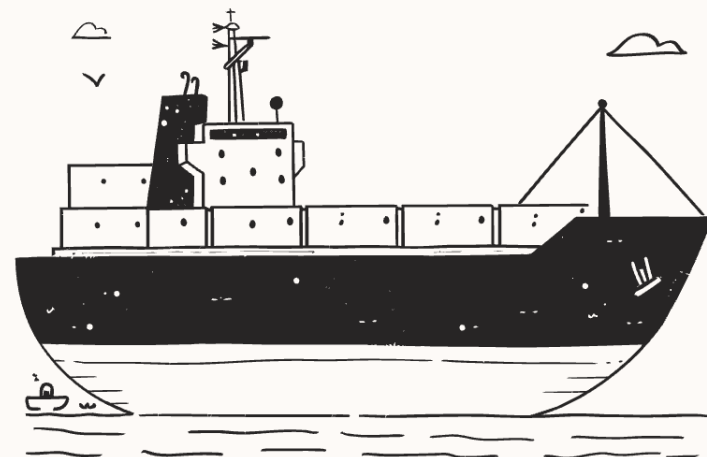
Неизменяемый шаблон с приложением и зависимостями

Слои (Layers)

Каждая инструкция Dockerfile создаёт новый слой

Контейнер

Запущенный экземпляр образа с изолированной файловой системой



Хранение образов

После сборки образ нужно где-то хранить и откуда-то получать при деплое. Для этого используются реестры образов — публичные и приватные.



Docker Hub

Публичный реестр образов от Docker. Бесплатный для публичных репозиторий. Идеален для старта и open-source проектов.



Private Docker Registry

Собственный реестр, развёрнутый на вашем сервере. Полный контроль над образами, без ограничений публичного хаба.



Nexus

Универсальный менеджер артефактов. Поддерживает Docker, Maven, npm и другие форматы. Подходит для корпоративных сред.

Docker Compose

`docker-compose.yml`

Файл описывает все сервисы приложения, их зависимости и конфигурацию. Запуск всего стека — одной командой:

```
docker-compose up -d
```

Compose автоматически создаёт сеть между контейнерами и управляет порядком запуска через `depends_on`.

ports

Проброс портов хоста в контейнер

volumes

Монтирование данных и конфигов

env

Переменные окружения для сервисов

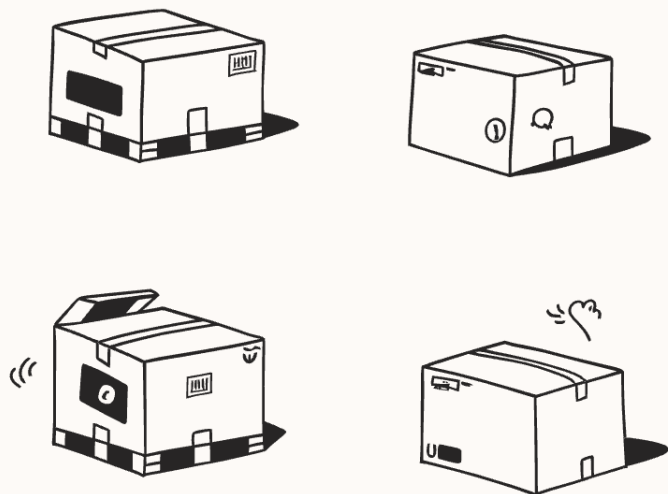
Pipeline

Интеграция Compose в CI/CD pipeline



Дополнительное ПО

После запуска приложения в контейнере необходимо обеспечить безопасный доступ из интернета через доменное имя и HTTPS.



nginx

Высокопроизводительный веб-сервер и обратный прокси



Reverse Proxy

Перенаправляет запросы к нужному контейнеру по пути или домену



Домен

Привязка доменного имени к IP-адресу сервера



SSL / HTTPS

Сертификат Let's Encrypt, шифрование трафика, безопасное соединение

Мониторинг



Мониторинг позволяет отслеживать состояние сервера и приложения в реальном времени. Используйте **Zabbix** или **Prometheus** для сбора метрик.

CPU и RAM

Загрузка процессора и использование оперативной памяти

Диски

Свободное место, IOPS, скорость чтения/записи

Доступность

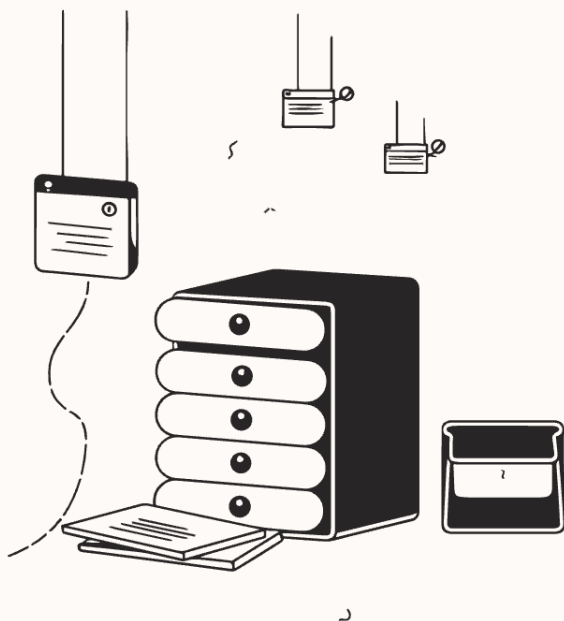
Проверка работоспособности сервиса (uptime, HTTP-статус)

Алерты

Уведомления при превышении порогов — email, Telegram, Slack

Логирование

Централизованное логирование позволяет собирать логи со всех сервисов в одном месте, выполнять поиск и анализировать ошибки.



ELK Stack

Elasticsearch — хранение и поиск

Logstash — сбор и обработка

Kibana — визуализация и дашборды

Loki + Grafana

Лёгкая альтернатива ELK. Loki собирает логи, Grafana отображает их. Меньше ресурсов, проще настройка.

Запросы к логам

Поиск по ключевым словам, фильтрация по времени, уровню (ERROR, WARN), сервису и хосту

Поиск ошибок

Быстрое обнаружение исключений, трассировок стека и аномалий в потоке логов

Документация

Хорошая документация — это то, что отличает профессионала от любителя. Она должна отвечать на все практические вопросы команды и будущего вас.

Документируйте всё, что делаете: каждый шаг развёртывания, каждую нестандартную настройку, каждое решение типовой проблемы.



Как развернуть

Пошаговая инструкция с нуля: зависимости, конфиги, команды запуска



Как запустить

Команды для старта сервисов, проверки статуса и первичной диагностики



Как обновить

Процедура обновления: pull, build, restart, rollback при ошибке



Смотреть логи

Где находятся логи, как их читать, какие команды использовать



Типовые проблемы

FAQ: частые ошибки, их причины и способы устранения

Kubernetes

Kubernetes — система оркестрации контейнеров. Для обучения используйте **Minikube** (локальный кластер) или **K3s** (лёгкий дистрибутив для сервера).



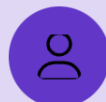
Deployment

Описывает желаемое состояние: сколько реплик запустить, какой образ использовать



Service

Стабильный сетевой адрес для доступа к подам внутри и снаружи кластера



Ingress

Маршрутизация HTTP-трафика по доменам и путям к нужным сервисам



Мониторинг

Prometheus + Grafana для метрик кластера, Loki для логов подов



СПАСИБО ЗА ВНИМАНИЕ



TG <https://t.me/linautonet>

TG чат <https://t.me/linautonetchat>

YouTube <https://www.youtube.com/@solizarevich>

VK <https://vk.com/linautonet>

Rutube <https://rutube.ru/channel/25269092>

Сайт <https://solizarevich.github.io>